

Automatisk Nummerplade Registration

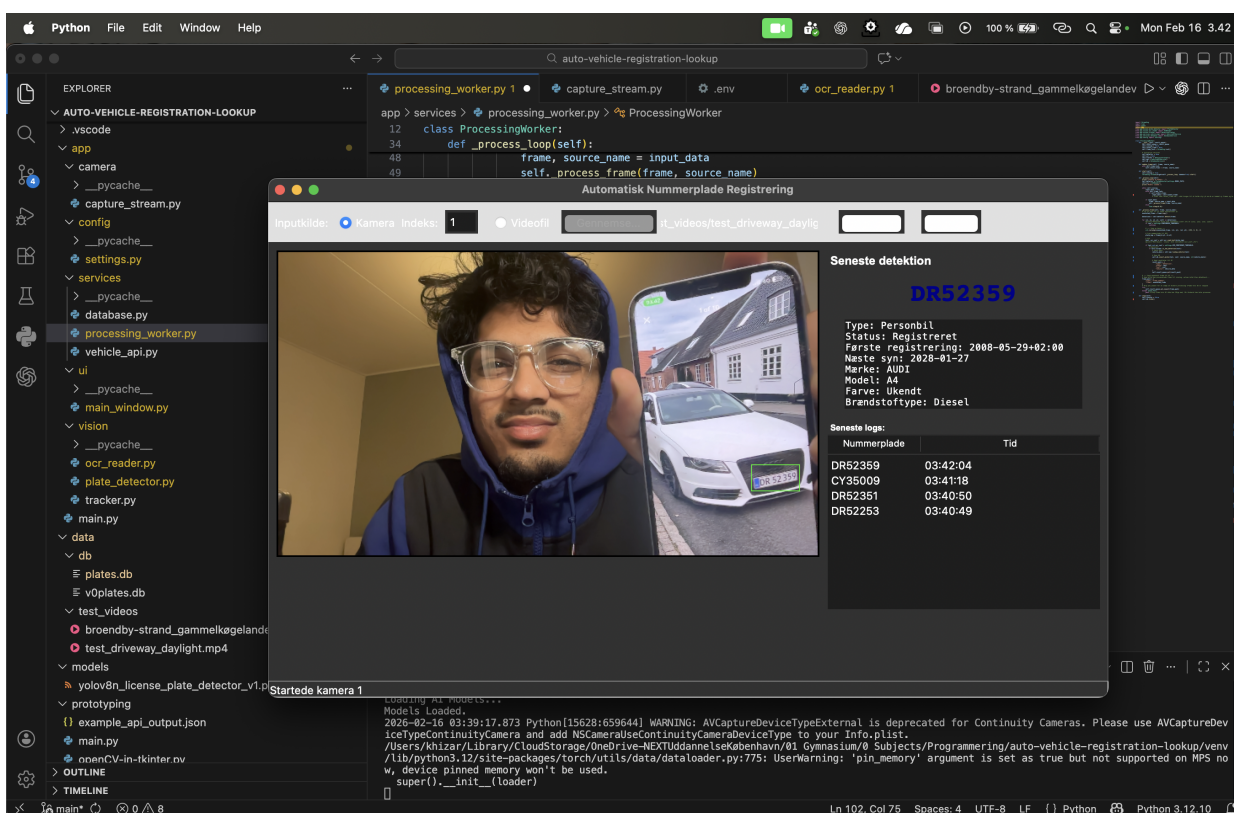
Programmering B

Khizar Irshad Chaudhry, 3.w

12. september 2026

Resumé

Dette projekt omhandler udviklingen af en Pythonbaseret desktopapplikation til automatisk nummerplade genkendelse. Systemet anvender YOLO til objekt-detektion, EasyOCR til tekstgenkendelse og foretager opslag via MotorAPI, hvorefter data gemmes i en SQLite database og vises i en Tkinter GUI.



Figur 1: ET OVERBLIK AF APPLIKATIONEN.

1 Indledning

I dag anvendes et system til automatisk nummerpladegenkendelse på de danske motorveje, parkeringssystemer, overvågning og trafikstyringen på færdslen. Dette system er også bredt kendt som ALPR, Automatic License Plate Recognition. ALPR systemer kombinerer teknologier som computer-vision, OCR (optical character recognition) og databaser for at automatisere identifikation af køretøjer gennem nummerplader.

Formålet med dette projekt er netop at konstruere et lignende system som ALPR, og dermed udvikle en Python baseret desktopapplikation, der kan automatisk:

1. Læse video fra kamera (live) eller video fil
2. Detektere nummerplader vha. AI model
3. Genkende teksten vha. OCR
4. Normalisere og validere danske nummerpladeformat
5. Foretag data indhentning tilknyttet til nummerpladen vha. API
6. Gemme resultater i database
7. Visualisere det hele i en view-layer vha. Tkinter GUI

For at fuldføre projektet kombineres flere teknologier, herunder bl.a.:

1. OpenCV
2. Tkinter
3. Ultralytics YOLO
4. EasyOCR
5. SQLite
6. REST API kommunikation

1.1 Problemformulering

Dette fører os videre til problemformuleringen, der skal besvares med den færdige applikation. **Hvordan kan man designe og implementere en Python applikation, der vha. computer vision, OCR, API og databaser automatisk kan identificere og registrere danske nummerplader på færdslen i realtid.**

2 Kravspecifikation

Projektet er udviklet efter kravene for applikationsforløbet. Sammen med den tidsbegrænset krav på ca. 2,5 indeholder det også følgende krav:

- Et 3-lags arkitektur (præsentations-, logik- og datalag).
- I præsentations- og logik-laget skal der implementeres en MVC pattern (Model View Controller). View og Controller skal være i præsentationslaget, og Model i logik-laget.
- View og Controller implementeres vha. Tkinter modulet i Python.
- Datalaget kan enten benytte en API eller database for at tilgå nødvendigt data fra data-laget.

Herunder er det vigtigt at understrege, det er ikke et krav at de enkelte dele kan sammenstykke til en velfungerende applikation, men det er acceptabelt hvis applikation består af separate fungerende dele.

Krav til denne dokumentation indeholder:

- Et use-case diagram til at beskrive brugerens interaktion med præsentationslaget.
- Et klasse-diagram med korrekt afbildning og relationerne imellem klasserne.
- Et eller flere flowcharts over de relevante selv udvalgt kode
- Mindst en selv designet abstrakt illustration, der hjælper en med at forstå en del af koden.

Med udgangspunkt i de generelle krav for applikationsforløbet, har dette projekt specificeret kravene yderligere, og hvordan de skal opfyldes. Dette ses nedenfor

2.1 Projektarkitekturkrav

Applikationen følger en 3-lags arkitektur

2.1.1 Præsentationslaget

Denne lag præsentere det nødvendige til brugeren, hvilket inkluderer, men er ikke begrænset til

- Tkinter GUI
- View + Controller (MVC)
- Live videovisning
- Plate detection
- Resultatpanel
- Logs visning

2.1.2 Præsentationslaget

Logiklaget sørge for alt processeringen, dette inkludere

- Nummerplade detection med YOLO model
- OCR-behandling med EasyOCR
- Validering og normalisering af formatet
- Deduplication (ingen dublikeret nummerplader)
- API request processen
- Databehandling

2.1.3 Datalaget

Datalaget omhandler alt operation med data. Dette inkludere

- Data gem (SQLite Database)
- API indhentning (MoterAPI)

2.2 Funktionelle krav (Need to have)

Denne sektion uddyber og specificere diverse funktionelle krav i detalje.

1. Systemet skal kunne læse video fra:
 - Kamera
 - Videofil
2. Systemet skal kunne:
 - Detektere nummerplader med YOLO modeller
 - Udtrække tekst med OCR
 - Normalisere teksten og validere dansk nummerplade format (2 bogstaver + 5 tal)
3. System skal:
 - Indhent data ift. nummerpladen via MoteAPI
 - Gemme det indhentet data samt nummerpladen og tidspunktet i SQLite database
 - Vise informationen i TkinterGUI
4. Der udover er det vigtigt at systemet er non-blocking (dvs. multithreaded), og forskellige aspekter af programmet kan køre asynkronous.

2.3 Ikke funktionelle krav

Derudover er det selvfølgelige preferable at have nedestående krav. Da det generalt øger brugertilfredsheden og applikation UX (user experience), og desto ikke mindre gøre det nemt at udvikle videre på applikationen.

- Realtidsbehandling
- Modulær arkitektur
- Skalerbar struktur
- Robust fejlhåndtering
- Performance optimering via cooldown

3 Computational Thinking

For at programmere systemer skal man have en helt særlig tilgang til problemløsning. Denne tilgang kaldes for computational thinking, og består af sine fire dele. Pattern Recognition, Abstraction, Decomposition, og Algorithm Design.

De fire grene af computational thinking anvendes naturligvis også programmet for dette projekt. I dette afsnit kommenteres der på disse aspekter.

3.1 Pattern Recognition

Pattern recognition, altså møster genkendelse er essentiel til udvikling af systemer og programmer. Igennem disse mønstre kodes programmer. Dog er møstergenkendelsen i dette program meget overkommelige og gentagne, der programmet gør brug af OCR og AI modeller (YOLO). Dette inkludere systemets genkendelse af følgende mønstre,

- **Nummerplade detektion**

Alle nummerplader detekteres via samme YOLO-model.

Yolo-modellen er gemt som [models/yolov8n_license_plate_detector_v1.pt](#), modellen er ikke trænet selv, men indhentet fra datascience siden [Kaggle](#). Måden YOLO-modellen fungerer på er,

hele billede frame behandles som et enkelt billed, der forward passes gennem det neurale netværk i [yolov8n_license_plate_detector_v1.pt](#). Billedet opdeles internt i et gitter, og for hver celler i gitteret forudsiger modellen om det givne trænet objekt findes (altså nummerpladen). Hvis objektet findes, resultateres det med objektets bounding box ([x](#), [y](#), [width](#), [height](#)) samt en confidence score.

Dette sker i objektet [PlateDetector](#) som anvendes i [ProcessingWorker](#). Metoden [PlateDetector.detect\(frame\)](#) benyttes, og det er denne methods output, der ser sådan ud:

Listing 1: YOLO modellen appender detectionen til en dictionary som output.

```
[
  {
    "bbox": [x1, y1, x2, y2],
    "confidence": 0.87
  }
]
```

Dette er et klassisk eksempel på pattern recognition, da alle objekter behandles ens. Dvs. samme algoritme, filtrering og outputsstruktur benyttes til alle nummerplade. Dette gøre logikken general og modulært.

- **OCR-mønster**

Tekst filtreres og valideres efter DK-format vha. OCR. Når YOLO modellen finder en bounding box, og dets egenskaber bliver *appendid* til dictionary, sendes det videre til objektet [OCRReader](#). Denne klasse har method [.read_text\(cropped_image\)](#), hvilket igen bruges i [ProcessingWorker](#). OCR returnere en liste af tekstforslag samt en confidence rating. Dette sker efter teksten normaliseres (store bogstaver, fjerner mellemrum samt specialtegn), og valideres efter det generelle nummerplade struktur i Danmark vha. regex:

Listing 2: Regex til validering af danske nummerplader

```
r"^[A-Z]{2}[0-9]{5}$"
```

Alt i alt returnere [OCRReader.read_text\(cropped_image\)](#), følgende ens resultat for alle valideret nummerplader med en confidence rating over den i [app/config/settings.py](#):

Listing 3: Output fra [OCRReader.read_text](#)

```
[
  ("CA49417", confidence_score),
  ("CA49A17", confidence_score)
]
```

- **API-struktur:**

Efter at returnere en valid nummerplade, benyttes MotorAPI til indhentning af data ift. nummerpladen. Dette gøres med metoden [VehicleAPIService.lookup_vehicle\(plate\)](#) API responsen er i et JSON-objekt format med standardiseret struktur. Nedenfor ses et eksempel på dette mønster:

Listing 4: JSON-respons fra MotorAPI med køretøjsdata

```
{
  "registration_number": "CA49417",
  "status": "Registreret",
  "type": "Personbil",
  "first_registration": "2014-10-03",
  "make": "CITROEN",
  "model": "GRAND_C4_PICASSO",
  "color": "Gra",
  "fuel_type": "Diesel",
  "mot_info": {
    "next_inspection_date": "2026-09-04"
  }
}
```

```
}  
}
```

3.2 Decomposition

Idéen bag decomposition er at dekomponere programmet op, hvilket er essentielt for at få en struktureret tilgang til at håndtere problemerne. Dette involverer bl.a. at opdele programmets problem/dele i mindre håndterbare dele. Det ses at allerede i slutningen af denne indledning, begyndte dekomponeringen af programmet. Systemets hovedkomponenter kan opdeles i følgende hovedkomponenter:

- **MainWindow (UI):** Håndterer brugergrænsefladen og visning af data.
- **CaptureStream (videokilde):** Administrerer adgang til kamera eller videofil.
- **ProcessingWorker (logik):** Styrer workflowet og trådhåndtering for at undgå blokering af UI.
- **PlateDetector (YOLO):** Modul dedikeret til objekt-detektion af nummerplader.
- **OCRReader (tekstgenkendelse):** Ekstraherer tekst fra de klippede billeder af nummerplader.
- **DetectionTracker (cooldown):** Holder styr på detekterede plader for at undgå duplikater.
- **VehicleAPIService (API):** Håndterer kommunikation og dataudveksling med MotorAPI.
- **DatabaseService (SQLite):** Sørger for lokal lagring og logning af opslag.

Her har hver klasse et klart ansvar, og udgøre en del af programmets helhed.

3.3 Abstraction

Abstraction indebærer at forenkle ting og fjerne irrelevante detaljer. Programmets kode anvender stærkt OOP (object-oriented programmering), et bredt anvendt programmeringsparadigme indenfor imperativt programmering. Nogle af disse komplekse dele der abstraheres bag klasser er:

- **YOLO-model** kapsles i `PlateDetector`
- **OCR-logik** kapsles i `OCRReader`
- **HTTP-requests** kapsles i `VehicleAPIService`
- **SQL-forespørgsler** kapsles i `DatabaseService`

Brugeren interagerer kun med GUI, og derfor forenkles alle disse funktioner bag deres hhv. klasse.

3.4 Algorithm Design

Algorithm design er nok det vigtigste element i programmering. Dette indebærer at udvikle trinvisse løsninger for at løse et problem. I dette program er hovedalgoritmen abstraheret bag klassen `ProcessingWorker` i filen `app/services/processing_worker.py`. Hovedalgoritmen i `processingworker.py` gøre følgende workflow:

1. Modtag frame
2. Kør YOLO
3. Filtrer på confidence
4. Crop nummerplade
5. OCR
6. Valider format

7. Tjek cooldown
8. API request
9. Database insert
10. Send resultat til UI

4 Udviklingsprocessen

Projektet blev udviklet iterativt, og mange protyper blev kodet af de enkeltvise elementer, før det fungeret som helhed. Dette reflekteres også i projektets mappe [prototyping](#). Denne iterativ udvikling af projektet så nogle gange sådan ud:

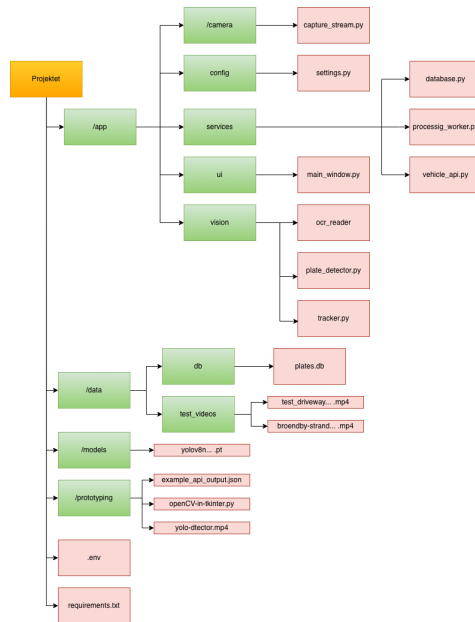
1. Opsætning af Tkinter GUI
2. Integration af OpenCV
3. Implementering af YOLO-model
4. OCR-integration
5. API-integration
6. Databaseimplementering
7. Multithreading
8. Deduplication-logik
9. UI-forbedringer

Git blev anvendt til versionsstyring, men med begrænset commit frekvens. GitHub adgang kan gives ved kontakt på Microsoft Teams.

5 Dokumentation af Programmet

5.1 Hierarki chart

På figur 2 ses projektets hierarki chart, der viser strukturen over filer og mapperne. Hver mappe repræsenterer et funktionelt område af programmet.



Figur 2: Hierarki chart over programmet.

5.2 Overordnet arkitektur

Programmet er opbygget efter en 3 lags arkitektur som krævet i applikationsforløbet:

1. Præsentationslag

- **MainWindow (Tkinter GUI):** Den primære brugerflade.
- Håndterer brugerinput.
- Viser videofeed i realtid.
- Viser hentede køretøjsdata.
- Viser log over tidligere registreringer.

2. Logiklag

- **ProcessingWorker:** Styrer programmets workflow og trådhåndtering.
- **PlateDetector:** Forestår objektgenkendelse (YOLO).
- **OCRReader:** Konverterer billeddata til tekst.
- **DetectionTracker:** Håndterer cooldown og forhindrer duplikater.
- **VehicleAPIService:** Kommunikerer med eksterne opslagstjenester.

3. Datalag

- **DatabaseService:** Interface til håndtering af lagring.
- **SQLite database (plates.db):** Persistent lagring af registreringer.

Klassediagram

Klassen `ProcessingWorker` er programmets centrale logikkomponent. Den fungerer som controller i logiklaget og er ansvarlig for at:

- Modtage frames
- Udføre nummerpladedetektion
- Udføre OCR
- Validere tekst
- Håndtere deduplication (cooldown)
- Foretage API opslag
- Indsætte data i database
- Returnere resultater til UI via en queue

Klassen kører i en separat tråd for at sikre, at GUI ikke blokeres under de tunge beregninger.

Konstruktor

Listing 5: Konstruktor for `ProcessingWorker`

```
def __init__(self, result_queue):
```

Parameter:

- **result_queue** (`queue.Queue`): Bruges til kommunikation mellem processing tråden og GUI tråden.

Funktion: Konstruktøren initialiserer de nødvendige hjælpeklasser: `PlateDetector`, `OCRReader`, `DetectionTracker`, `VehicleAPIService` og `DatabaseService`. Disse objekter oprettes som instans variabler og bruges i hele klassens levetid som objekt.

Metode: `start()`

```
def start(self):
```

Funktion: Starter worker tråden. Den opretter en intern tråd og kalder den private metode `_run()`. Formålet er at gøre klassens proces asynkron, så brugerfladen forbliver responsiv.

Metode: `stop()`

```
def stop(self):
```

Funktion: Stopper worker tråden kontrolleret ved at sætte en intern flag variabel. Dette sikrer en korrekt afslutning af programmet uden at efterlade aktive "zombie" tråde.

Metode: `update_frame(frame, source_info)`

```
def update_frame(self, frame, source_info):
```

Parametre:

- `frame` (`numpy.ndarray`): Billeddata fra OpenCV.
- `source_info` (`str`): Information om inputkilden (f.eks. kameranavn eller filsti).

Funktion: Gemmer det seneste frame, som skal behandles. Metoden kaldes fra GUI tråden, men selve data behandlingen sker i worker tråden.

Metode: `_run()`

```
def _run(self):
```

Funktion: Intern metode (hovedloop), der kører kontinuerligt, så længe systemet er aktivt. Den henter det seneste frame, kalder `_process_frame(frame)` og sover kortvarigt for at undgå unødigt CPU overbelastning.

Metode: `_process_frame(frame)`

```
def _process_frame(self, frame):
```

Funktion: Denne metode indeholder hele systemets pipeline i ni trin:

1. YOLO detektion via `PlateDetector`
2. Filtrering baseret på confidence score
3. Crop af det relevante nummerpladeområde
4. OCR ekstraktion via `OCRReader`
5. Validering af tekstformat (Regex)
6. Cooldown check via `DetectionTracker`
7. Opslag af køretøjsdata via `VehicleAPIService`
8. Lagring af hændelsen via `DatabaseService`
9. Afsendelse af det samlede resultat til GUI via `result_queue`

Samspil med Andre Klasser

`ProcessingWorker` fungerer som bindeleddet i systemet og anvender følgende objekter:

- **`PlateDetector`** → returnerer bounding boxes.
- **`OCRReader`** → returnerer rå tekst.
- **`DetectionTracker`** → forhindrer redundante registreringer.
- **`VehicleAPIService`** → returnerer tekniske data i JSON format.
- **`DatabaseService`** → sikrer persistent lagring i SQLite.

6 Pseudokode

Denne pseudokode nedenunder repræsenterer hele systemets logiske pipeline.

Listing 6: Pseudokode for systemets processerings pipeline

```
function process_frame(frame):
    detections = plate_detector.detect(frame)

    for detection in detections:
        if detection.confidence > threshold:
            plate_crop = crop(frame)
            text = ocr_reader.read_text(plate_crop)

            if valid_format(text):
                if tracker.is_new_detection(text):
                    vehicle_data = api.lookup_vehicle(text)
                    database.insert_plate(text, vehicle_data)
                    send_detection_to_ui(text, vehicle_data)
```

7 Udvalgt kodelykke med kommentar

Formatvalidering med Regex

For at sikre, at systemet kun arbejder videre med korrekte data, anvendes Regular Expressions (Regex) til at validere de aflæste nummerplader.

Listing 7: Python-funktion til validering af nummerpladeformat

```
import re

def is_valid_plate(plate_text):
    pattern = r"^[A-Z]{2}[0-9]{5}$"
    return re.match(pattern, plate_text) is not None
```

Forklaring

Dette specifikke regex-mønster sikrer, at inputtet følger den danske standard for personvogne:

- `^` : Angiver starten på strengen.
- `[A-Z]{2}` : Matcher præcis to store bogstaver (A-Z).
- `[0-9]{5}` : Matcher præcis fem tal (0-9).
- `$` : Angiver slutningen på strengen.

Ved at bruge både `^` og `$` sikrer vi, at funktionen returnerer **False**, hvis der er ekstra tegn eller støj i teksten, hvilket er en vigtig del af fejlhåndteringen i OCR processen.

7.1 Test

For at sikre at applikationen fungerer som forment udøfres diverse test på applikationen.

For at efterprøve systemets robusthed blev følgende tests udført:

1. Videoinput test

Der er foretaget omfattende tests med `.mp4`-filer for at sikre korrekt frame læsning. For at skaffe realistisk testdata blev der foretaget videooptagelser på *Gammel Køge Landevej i Brøndby Strand*. Disse optagelser indeholdt biler i bevægelse, hvilket tillod test af systemets evne til at håndtere skiftende vinkler og hastigheder.

2. Kamera test:

Live capture er testet via OpenCV for at sikre, at systemet kan streame og behandle billeder i realtid uden mærkbar forsinkelse (latency).

3. OCR og Model test:

Systemet er blevet testet under forskellige lysforhold og afstande. For at optimere præcisionen blev der ”leget” med *confidence thresholds*. Under denne fase blev systemet testet på billeder af **biler til salg på Facebook**, da disse ofte præsenterer udfordrende vinkler og varierende billedkvalitet.

4. API test:

Det er verificeret, at **VehicleAPIService** korrekt modtager og parser JSON-responser fra MotorAPI, så de tekniske data kan præsenteres korrekt i UI.

5. Database test:

Der er foretaget kontrol af, at alle registreringer indsættes korrekt i SQLite-databasen (**plates.db**), og at data er persistente efter genstart af programmet.

8 Diskussion

Udviklingen af dette projekt har været en teknisk udfordrende og lærerig proces. Dog fyldt med glæde og passion. Projektet kombinerer flere forskellige teknologier, *computer vision*, OCR, API kommunikation, databasehåndtering og GUI programmering, hvilket gør systemet komplekst, men også realistisk i forhold til moderne softwareløsninger.

En af de største udfordringer har været at sikre, at systemet forblev responsivt. Da YOLO modellen og OCR behandling er beregningstunge operationer, ville tkinter GUI'en tit fryse til, hvis behandlingen skete i hovedtråden. Dette blev løst ved at implementere **multithreading** og anvende en **queue.Queue** til sikker kommunikation mellem tråde.

En anden udfordring har været OCR nøjagtighed. EasyOCR kan meget tit fejllæse tegn ved lav opløsning, dårlige lysforhold eller bevægelse i billedet.

Ydelsesmæssigt fungerer systemet stabilt ved normal brug, men ved høj frame rate eller mange samtidige detektioner kan CPU belastningen stige markant. YOLO er især ressourcekrævende. Systemet kunne forbedres ved:

- GPU-acceleration.
- Mere avanceret *object tracking*.

Arkitekturmæssigt følger systemet en **3-lags struktur** med elementer fra **MVC** i præsentations- og logiklaget. Dette gør koden mere overskuelig og skalerbar. Hvis projektet skulle videreudvikles, kunne man overveje en *client-server* arkitektur, hvor vision og API kald håndteres af en backend server, og GUI fungerer udelukkende som klient.

Samlet set demonstrerer projektet både styrker og realistiske begrænsninger ved realtids *computer vision* systemer.

9 Konklusion

Formålet med projektet var at udvikle en Python baseret desktopapplikation, der automatisk kan identificere og registrere danske nummerplader i realtid ved hjælp af *computer vision*, OCR og API integration.

Problemformuleringen lød:

Hvor- dan kan man designe og implementere en Python applikation, der vha. computer vision, OCR, API og databaser automatisk kan identificere og registrere danske nummerplader på færdslen i realtid?

Projektet viser, at dette kan opnås ved at kombinere:

- Deep learning baseret objekt-detektion (YOLO).

- OCR tekstgenkendelse (EasyOCR).
- Regex validering.
- REST API integration via MotorAPI.
- Standardiseret JSON databehandling.
- SQLite databasepersistens.
- Multithreaded arkitektur.

Systemet opfylder de opstillede funktionelle og arkitektoniske krav. Dette demonstrerer derfor også forståelse for:

- Objektorienteret programmering.
- MVC og 3-lags arkitektur.
- Computational Thinking.
- API kommunikation og databasehåndtering.

Applikationen fungerer stabilt under normal brug og viser, hvordan flere teknologier kan integreres i ét samlet system. Samtidig illustrerer projektet de praktiske udfordringer ved realtidsbehandling og afhængighed af eksterne tjenester. Projektets mål er derfor opnået, og problemformuleringen er besvaret.